

Atividade 01

01_tokens:

```
import java.util.LinkedHashMap;
import java.util.Map;

public class Main {
    static final String ALUNO = "Camila Reinke";
    static final String RA = "24001081-2";

    static final String ENTRADA = "resultado = somaAnterior - valor / 100;";

    private static final Map<Character, String> SIMBOLOS = new LinkedHashMap<>();

    static {
        SIMBOLOS.put('=', "ATRIBUICAO");
        SIMBOLOS.put('+', "SOMA");
        SIMBOLOS.put('-', "SUBTRACAO");
        SIMBOLOS.put('*', "MULTIPLICACAO");
        SIMBOLOS.put('/', "DIVISAO");
        SIMBOLOS.put('.', "PONTO_E_VIRGULA");
        SIMBOLOS.put('(', "ABRE_PARENTESES");
        SIMBOLOS.put(')', "FECHA_PARENTESES");
    }
}
```

ALUNO: Camila Reinke | RA: 24001081-2
ENTRADA: resultado = somaAnterior - valor / 100;

LEXEMA	TOKEN

resultado	IDENTIFICADOR
=	ATRIBUICAO
somaAnterior	IDENTIFICADOR
-	SUBTRACAO
valor	IDENTIFICADOR
/	DIVISAO
100	INTEIRO
;	PONTO_E_VIRGULA

Atividade 02

02_lexico:

```
import java.util.Arrays;
import java.util.HashSet;
import java.util.Set;

public class Main {
    static final String ALUNO = "Camila Reinke";
    static final String RA = "24001081-2";

    static final String IDENTIFICADOR_PERSONALIZADO = "nota_cr";

    static final Set<String> PALAVRAS_RESERVADAS = new HashSet<>() {
        {
            Arrays.asList("int", "if", "else", "while", "return")
        }
    };

    public static void main(String[] args) {
        System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
        executarTeste("TESTE A - entrada compacta",
            "int total=valor+10;");
        executarTeste("TESTE B - espaços e comentário",
            "int total = valor + 10; // este comentario sera ignorado");
        executarTeste("TESTE C - reservadas e identificadores",
            "int inteiro = 0; while (inteiro < intValue) inteiro = inteiro + 1;");
        executarTeste("TESTE C2 - identificador personalizado",
            "int " + IDENTIFICADOR_PERSONALIZADO + " = 10;");
        executarTeste("TESTE D - caractere inválido",
            "int valor# = 1;");
    }
}
```

inteiro	LETRA	IDENTIFICADOR
+	SIMBOLO	SOMA
1	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

=== TESTE C2 - identificador personalizado ===
ENTRADA: int nota_cr = 10;

LEXEMA	CLASSE	TOKEN

int	LETRA	PALAVRA_RESERVADA
nota_cr	LETRA	IDENTIFICADOR
=	SIMBOLO	ATRIBUICAO
10	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

=== TESTE D - caractere inválido ===
ENTRADA: int valor# = 1;

LEXEMA	CLASSE	TOKEN

int	LETRA	PALAVRA_RESERVADA
valor	LETRA	IDENTIFICADOR
#	DESCONHECIDO	ERRO_LEXICO
=	SIMBOLO	ATRIBUICAO
1	DIGITO	INTEIRO
;	SIMBOLO	PONTO_E_VIRGULA

Atividade 03

03_erro_compilador:

```

public class Main {
    static final String ALUNO = "Camila Reinke";
    static final String RA = "24001081-2";

    public static void main(String[] args) {
        System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);

        int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e virgula

        if (total > 20 { // CORRIGIR 2: feche corretamente o parenteses
            System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto
            System.out.println("Resultado: " + total);
        }
    }
}

```

Main.java:8: error: ';' expected
 int total = 10 + 20 // CORRIGIR 1: acrescente o ponto e virgula
 ^

Main.java:10: error: ')' expected
 if (total > 20 { // CORRIGIR 2: feche corretamente o parenteses
 ^

Main.java:11: error: unclosed string literal
 System.out.println("Programa corrigido!"); // CORRIGIR 3: feche o texto entre aspas
 ^

3 errors
 Debug this error

04_programa_corrigido:

```

public class Main {
    static final String ALUNO = "Camila Reinke";
    static final String RA = "24001081-2";

    public static void main(String[] args) {
        System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);

        int total = 10 + 20;

        if (total > 20) {
            System.out.println("Programa corrigido!");
            System.out.println("Resultado: " + total);
        }
    }
}

```

ALUNO: Camila Reinke | RA: 24001081-2
 Programa corrigido!
 Resultado: 30

Atividade 04

05_descendente:

```

public class Main {
    static final String ALUNO = "Camila Reinke";
    static final String RA = "24001081-2";

    static final String ENTRADA = "(2 + 3) * 4";
    static final boolean DEMONSTRAR_RECURSAO_ESQUERDA = false;

    public static void main(String[] args) {
        System.out.println("ALUNO: " + ALUNO + " | RA: " + RA);
        System.out.println("ENTRADA: " + ENTRADA);
        System.out.println();

        if (DEMONSTRAR_RECURSAO_ESQUERDA) {
            demonstrarRecursoEsquerda(0);
            System.out.println();
        }

        try {
            Parser parser = new Parser(ENTRADA);
            int resultado = parser.analisar();
            System.out.println();
            System.out.println("ENTRADA ACEITA");
            System.out.println("RESULTADO DA EXPRESSAO: " + resultado);
        } catch (ErroSintatico erro) {
            System.out.println();
            System.out.println("ERRO SINTATICO: " + erro.getMessage());
        }
    }
}

```

SAINDO de <factor> | lookahead = +
 SAINDO de <term> | lookahead = +
 Encontrado operador +
 Novo lookahead: 3
 ENTRANDO em <term> | lookahead = 3
 ENTRANDO em <factor> | lookahead = 3
 Numero reconhecido: 3
 Novo lookahead:)
 SAINDO de <factor> | lookahead =)
 SAINDO de <term> | lookahead =)
 SAINDO de <expr> | lookahead =)
 Novo lookahead: *
 Fechando parenteses
 SAINDO de <factor> | lookahead = *
 Encontrado operador *
 Novo lookahead: 4
 ENTRANDO em <factor> | lookahead = 4
 Numero reconhecido: 4
 Novo lookahead: \$FIM\$
 SAINDO de <factor> | lookahead = \$FIM\$
 SAINDO de <term> | lookahead = \$FIM\$
 SAINDO de <expr> | lookahead = \$FIM\$

ENTRADA ACEITA
 RESULTADO DA EXPRESSAO: 20

Atividade 05

06_ascendente:

```
import java.util.ArrayList;
import java.util.HashMap;
import java.util.List;
import java.util.Map;

public class Main {
    static final String ALUNO = "Camila Reinke";
    static final String RA = "24001081-2";

    static final String ENTRADA_VALIDA = "id+id*id";
    static final String ENTRADA_INVALIDA = "id*id";

    // Na primeira execução, use ENTRADA_VALIDA. Depois troque por ENTRADA_INVALIDA
    static final String ENTRADA = ENTRADA_VALIDA;

    static final Map<Integer, Map<String, String>> ACTION = new HashMap<>();
    static final Map<Integer, Map<String, Integer>> GOTO = new HashMap<>();
    static final Map<Integer, Regra> REGRAS = new HashMap<>();

    static {
        // Gramática:
        // 1) E -> E + T      2) E -> T
```

ALUNO: Camila Reinke | RA: 24001081-2
ENTRADA: id+id*id

PILHA	ENTRADA	ACAO

\$ 0	id+id*id\$	SHIFT id -> estado 5
\$ 0 id 5	+id*id\$	REDUCE F -> id
\$ 0 F 3	+id*id\$	REDUCE T -> F
\$ 0 T 2	+id*id\$	REDUCE E -> T
\$ 0 E 1	+id*id\$	SHIFT + -> estado 6
\$ 0 E 1 + 6	id*id\$	SHIFT id -> estado 5
\$ 0 E 1 + 6 id 5	*id\$	REDUCE F -> id
\$ 0 E 1 + 6 F 3	*id\$	REDUCE T -> F
\$ 0 E 1 + 6 T 9	*id\$	SHIFT * -> estado 7
\$ 0 E 1 + 6 T 9 * 7	id\$	SHIFT id -> estado 5
\$ 0 E 1 + 6 T 9 * 7 id 5	\$	REDUCE F -> id
\$ 0 E 1 + 6 T 9 * 7 F 10	\$	REDUCE T -> T * F
\$ 0 E 1 + 6 T 9	\$	REDUCE E -> E + T
\$ 0 E 1	\$	ACCEPT